

Negamax Alpha-Beta Pruning for Othello with Move Order and Transposition Caching

McGill Computer Science

Luca Louis

Abstract

This report presents a game-playing approach to Othello using a search tree to represent the pathway and move selection through a game. The primary goal is to design and implement the strongest agent possible in the given constraints, by trying to optimize both the creation and traversal of the search tree, of which critical importance is assigned to how each move in the tree is evaluated as favorable or not. This is accomplished through the selection of Heuristics that encode strategic knowledge of the game, and ultimately represent a kind of artificial intuition derived by attempting to mirror human cognitive processes. Individual heuristics are given integer weights and summed as a linear combination to ultimately give each considered move a total evaluative score. The strongest agent we found is an implementation of the negamax variant of the minimax search algorithm enhanced with alpha-beta pruning, iterative deepening, a transposition table for caching evaluated board states, and move ordering for arranging candidate moves in an order that places the most anticipated promising moves at the top. Our Heuristic evaluation function involves corner occupancy, edge control, mobility, stability and frontier discs, each weighted according to their relative significance on gameplay.

1. Introduction

In Game-playing AI algorithms, we leverage the unique combination of fundamentally relying on sheer computational brute force and using humanlike anti-computational intuitive strategies to produce the most efficient and deepest search possible. The allure of this domain in AI is precisely this revolvment around trying to explicitly articulate the human game-playing thought process, and finding a way to most equivalently transcribe that into code where its execution can then be carried out on a level of hardware that far surpasses our own. It's a cohesive approach that encompasses mathematical rigour, designing and abstracting creative strategies, and the less tangible attempt to encode into a machine what a lot of board game world champions describe as instinct or "feel". The reality of these two sides to the approach is abundantly clear, so much so we even explicitly see it in the template of our starter code: We have the method `step()` which represents the implementation of the brute searching through candidate moves; and we have the method `evaluate_board()` which contain the heuristics that attempt to distill immaterial strategic understanding into a computationally tractable form. It's a wholesome convergence between the human touch and machine precision that mutually depend on each other to produce the strongest performing agents. Our agent is a particular implementation that navigates this convergence elegantly, which we believe correctly identifies key strategic features of the Othello game and performs accordingly. There is equally an emphasis on maintaining good trade-offs between an appropriately comprehensive search depth when called for, and the accompanying efficiency to not explore redundant lines of moves as implemented by the move ordering and pruning techniques.

As with most two player board games, the challenge of searching moves in Othello lies in the exponential growth of possible game states as the game progresses. A significant portion of the strategic maneuvering

in Othello occurs in the middlegame where the branching factor typically hovers around 10 available moves per position. This means that even a relatively low search depth of 5 or 6 moves ahead quickly becomes infeasible in the imposed 2 second time limit. Without the optimizations of efficient search optimization, the raw volume of potential positions would overwhelm the agent's ability to perform meaningful analysis within this allotted time.

2. The Algorithm - Design Considerations and Methodology

One of the first things to understand about designing a high-performance agent is to understand to what extent tweaks in the evaluation function can result in performance changes, and whether large performance disparities between head-to-head agent matchups are a true indication of "strength" in the broader domain of Othello solving. While presumably this question is equally applicable to the search strategy, we came across this curious point more with respects to the heuristic developments, which is thus the angle we will elaborate on.

Othello is a deterministic game which means decision computations are without an element of chance - agents follow a fixed decision-making process based on their programmed heuristics. Consequently, when two agents with similar heuristic structures compete, even minor enhancements in one agent's evaluation function can lead to significant result sways. For instance, a direct copy of `gpt_greedy_corners_agent` into a new agent that merely adds a greedy edge function in addition to the pre-existing greedy corners strategy leads to close to a 100%-win rate - with only 3 extra lines of code. However, this pronounced improvement does not necessarily indicate a broadly stronger agent. **The enhanced agent's success is often specific to the limitations of its predecessor** (`gpt_agent` in this example but this case can be generalized). While such enhancements can yield impressive results in controlled comparisons, they may not translate to universal superiority across a diverse competitive scenario. Not only may the performance gain against opponents with different heuristics be less pronounced, but in some cases versus opponents with entirely different evaluative structures, the enhanced agent may actually perform worse than its predecessor which it is ostensibly "stronger" than. Understanding and addressing these cyclical performance relationships when developing new agents is critical, necessitating robust and generalized performance gains rather than exploiting specific opponent weaknesses that can result in the intransitive performance we often witnessed down the line of developing later agents.

Our task when attempting to develop stronger heuristic functions is thus to comprehensively address all aspects of the game's strategic landscape, ensuring that our agent does not overlook any critical factors that could be exploited by competing agents (especially in the context of a tournament with head-to-head matchups). This means analyzing and integrating a wide range of heuristics that cover various gameplay scenarios, from corner control and edge stability to mobility and frontier dynamics. By doing so, we hopefully create a robust and versatile evaluation framework that leaves little room for adversaries to find simple exploits or weak points. Additionally, this issue speaks to the importance of continuously testing our agents against a diverse set of opponents with varying strategies to identify and rectify any unforeseen vulnerabilities. As such, our methodology involved creating many simple baseline agents (iterated modifications of `gpt_agent`) to provide these comparative benchmarks. This approach ensures that any new agent aiming to outperform ours must possess genuinely superior strategic capabilities (to which we will tip our hats), rather than merely capitalizing on minor, trivial oversights.

3. The Algorithm – Heuristic Evaluation Function

While it's probably more conventional in the AI literature to discuss the search strategy before the evaluation function, we have just discussed the nuances of designing an evaluation function at length and so feel it more naturally to exposit this function now rather than later. Furthermore, the way the algorithm evaluates the game state is arguably more significant to then how it traverses the possible states in order to find the best move, and most resources we consulted corroborate this.

The heuristics enable the computer to evaluate how good a position is for the player, so when the search simulates moves ahead, we can evaluate the hypothetical future board and backpropagate how favorable that move looks to the present game state. In Othello, we found that Corner Importance, Edge Importance, Risk of Corner-adjacent squares, Mobility, Stability, and Frontier dynamics were to be our chosen evaluative features:

3.1. Corner Occupancy

Corners in Othello are significant because disks occupying those squares are immune to being flipped for the remainder of the game – corner squares cannot be surrounded or “flanked”. Securing corners increases your stability in that region of the board, asserts control over surrounding squares and allows easier possibilities to branch out of the edges emanating from that corner, surrounding any opponents’ disks that are centralized. The code checks the four corner positions: (0,0), (0, N-1), (N-1, 0), and (N-1, N-1) to count how many are occupied by the player or the opponent. Each corner occupied by the player adds a positive weight to the evaluation score, while each corner occupied by the opponent subtracts from it.

3.2 Edge Occupancy

For similar reasons to corner occupancy, edge disks are harder to flip than interior disks, because they can only be outflanked across 1 dimension (horizontally or vertically depending on which edge) instead of 3 dimensions for interior disks (horizontally, vertically and diagonally). Edge positions are those where either the row or column index is 0 or N-1 (excluding corners). The code iterates through each identified edge position and increments the counts for number of disks for both the player and the opponent. The difference is computed and is multiplied by the weight.

3.3 Corner Adjacent

These are the positions diagonally adjacent to the corners, known as the “X-squares”. We give a penalty weight to occupying these squares as doing so prematurely can provide opponents with opportunities to capture the adjacent corners. For varied board sizes, the X-squares are (1, 1), (1, N-2), (N-2, 1), (N-2, N-2). The code checks if these X-squares are occupied by the player or the opponent, counts a tally, and multiplies by the negative edge weights.

3.4 Mobility

Mobility refers to the number of legal moves available to a player which is clearly favorable. A greater number of moves provides more flexibility and options, the heuristic also rewards limiting the opponent’s mobility, forcing them into making undesirable moves, or moves out of a lack of choice, rather than allowing them to make active good moves. High mobility enables the execution of strategies that can lead to capturing key areas like corners or edges. Mobility is calculated by determining the set of valid moves for both the player and the opponent by checking each empty board position to see if placing a disc there would capture opponent discs. We then subtract the number of opponents legal moves from the player’s legal moves and multiplied by the heuristic weight.

3.5 Stability

Stable discs are those that cannot be flipped for the remainder of the game, either because they are anchored by corners or edges or because they are already surrounded by disks and have been flipped by previous moves. Accumulating stable discs provides a lasting advantage, as they offer a secure foundation and immovable scores. We calculate the difference between the players and the opponent's stable discs, and as usual, multiply by the weight.

3.6 Frontier Disks

Frontier discs are those adjacent to empty squares, making them more susceptible to being flipped by the opponent. It must be adjacent to at least one empty square. Having fewer frontier discs reduces vulnerability and limits the opponent's opportunities to capture discs so its in the best interest of the player to have as few frontier discs as possible. The code iterates over the board to count the number of frontier disks each player has. The subtraction is performed, and the weight is multiplied to the result.

3.7 Win Bonus

The win bonus is used for terminal state prioritization. It's initialised as a substantial constant used to significantly amplify the evaluation score when the game reaches a terminal state in the search (neither player has any moves yet). This ensures that terminal positions are prioritized and overrides any heuristic evaluations derived from other board features. The method determines if the game has reached a terminal state by checking if both players have no valid moves yet, signifying the case where the game is over and the final disk count determines the winner. The disks are counted for both players and the disc difference is multiplied by the large win bonus giving the terminal state overwhelming priority in the evaluated linear combination.

3.8 Weighting Factors

Each heuristic is assigned a specific weight reflecting its strategic importance. The heuristic function `evaluate_board()` computes the linear combination of all the individual weights, which represents the evaluative score for each move under consideration. In the search function. The move with the highest score is returned and deemed the most favourable move in that game state. The weights were set based on intuitive insights for which features contribute most or least significantly to the game strategy, and then refined by trial and error. Weights ultimately prove significant in the overall performance of an agent and will form a large part of the discussion later on future improvements.

3.9 Unincluded heuristics

Over the course of development, a few other heuristics were considered and ultimately discarded for proving detrimental or ineffectual to the evaluation function. Once such attempt was a heuristic that measured center control. Center control refers to the strategic advantage gained by occupying the 4 center squares that form a 2x2 block in the middle of the board, along with the surrounding squares that provide pathways to these central positions. The rationale was that controlling the center allows a player to have more flexibility in making future moves. Central discs can flip a larger number of opponent discs in multiple directions, offering more opportunities for gaining territory. Occupying central discs in the opening can often result in stable disks later after moves get played and disks get dropped around them (assuming they are held on to). Center squares also provide even access to each corner, instead of having a majority of disks in one side only close to 2 corners. In practice, including this heuristic harmed the performance of the agent, I suspect all the advantages of central corner was already encompassed by our other heuristics like mobility, stability and to an extent, frontier disks. Additionally, greedily occupying center squares in the

opening does have a risk attached to it, if improperly managed it allows a lot of potential for flanking from all sides, and are thus vulnerable to being flipped often. Relating to this point, some of the literature even suggests having a majority disk difference in the opening is explicitly detrimental and advocates negative disk differential weights in the opening.

4. The Algorithm – Search Strategy

At the core, our search algorithm utilizes the Minimax algorithm, a decision paradigm that minimizes the loss in a worst-case scenario. In two-player games like Othello, Minimax assumes that both players play optimally: one aims to maximize the score (Max), while the other aims to minimize it (Min). The algorithm explores all possible moves, constructing a game tree where nodes represent board states and edges represent moves. As a result of the exponential progression of the tree often leading to the computational infeasibility of reaching terminal states, we evaluate nodes at the end of our search using a heuristic function allowing the algorithm to estimate the desirability of a game state without exhaustively exploring all possible future moves. The function assigns a numerical value to each terminal node, reflecting the favourability of that position for the player. These evaluations are then propagated back up the game tree, allowing the algorithm to select moves that maximize the player's advantage while minimizing the opponent's potential gains in the current game state.

While the foundational Minimax algorithm operates by separately handling the maximization and minimization phases for each player, our implementation adopts the Negamax variant. Negamax simplifies the algorithm by leveraging the zero-sum property of two-player games, wherein one player's gain is inherently the other player's loss. The algorithm assumes that the score from the perspective of the current player is the negative of the score from the opponent's perspective. By treating both players symmetrically, the algorithm is simplified slightly as it eliminates the need to handle separate cases for maximizing and minimizing players. The essence of the algorithm remains the same, focusing on maximizing the current player's advantage while minimizing the opponent's potential gains.

Seeing as we cannot exhaustively search all moves, it is in our interest to optimize the moves that we can search. By applying Alpha-Beta Pruning, we can intelligently eliminate large portions of the search tree that are guaranteed not to influence the final decision, thereby focusing our computational resources on the most promising moves. This optimization allows us to effectively narrow down the subset of moves we evaluate, ensuring that we prioritize high-impact moves while disregarding less optimal options early in the search process. Consequently, Alpha-Beta Pruning not only enhances the efficiency of our search algorithm but also enables us to delve deeper into the game tree within the allotted time, ultimately leading to more informed and strategic move selections despite the inability to explore all possible moves exhaustively. In alpha beta pruning, alpha represents the best already explored option along the path to the root for the maximizer, and beta the same for the minimizer. If the current node's score is greater than or equal to beta, no need to explore further siblings as the min player's optimal move will prevent us from reaching that hypothetical. In this case we can disregard that move and that branch. We also implement an iterative deepening approach. This approach involves performing a series of depth-limited searches, starting with a shallow depth and progressively increasing it. Iterative Deepening ensures that the agent can return the best move found within the available constraints of 2 seconds per move, even if deeper searches are incomplete because of this.

Additionally, we implement a dedicated move ordering method. Move Ordering is a technique aimed at arranging the sequence of moves evaluated by the search algorithm in a manner that maximizes the

effectiveness of Alpha-Beta Pruning. By prioritizing the exploration of the most promising moves first, the algorithm increases the likelihood of encountering pruning opportunities early in the search process, thereby reducing the number of nodes that need to be evaluated. In our agent, move ordering is achieved through a shallow evaluation of each potential move. Specifically, for each valid move, the agent performs a tentative execution of the move to generate a new board state. This state is then evaluated using a heuristic function that estimates the desirability of the position for the player. Moves are subsequently ranked based on these heuristic scores, with higher-scoring moves being prioritized for exploration. The combination of iterative deepening and move ordering ensures that the agent not only explores the game tree efficiently but also consistently improves its move selection quality as the search depth increases.

A last additional feature we use is a transposition table, a form of memoization used to store and retrieve the evaluation of previously encountered board states. This is realised by the fact that when looking ahead, different move orders can achieve the same position a couple moves down the search. In these cases, instead of re-evaluating the same position achieved through a slightly different sequence of moves, the identical board state and its evaluation is already stored in a hash table. Preventing redundant computations is invaluable in games with a lot of state repetition, which will typically occur in the opening phase, due to standardized and symmetrical moves with few disks on the board. In our implementation, the transposition table is realized through a hash table that maps a unique hash of each board state to its corresponding evaluation and the depth at which it was computed. The hashing process involves converting the board state into a byte representation and combining it with the current player's identifier to generate a unique key. This ensures that each unique state-player combination is accurately represented and retrievable. During the Alpha-Beta search, the agent first checks if the current board state exists in the transposition table with an adequate depth. If a valid entry is found, the stored evaluation is used directly, bypassing the need for re-evaluation.

5. The Code – Overview of method responsibilities and parameters

Method	Purpose	Parameters
step	Determines and returns the best possible move for the agent based on the current board state within a time limit.	Chess_board: current board state Player: identifier for agent Opponent: identifier for opponent
alpha_beta	Implements the Negamax variant of Minimax with Alpha-Beta Pruning to evaluate and select the optimal move recursively.	Board: current board state Player: identifier for agent Opponent: identifier for opponent Depth: remaining depth to search Alpha: best score for max Beta: best score for min Start_time: time stamp marking search start Time_limit: max search time (2s) Valid_moves: list of valid moves

order_moves	Ranks and orders the list of valid moves based on heuristic evaluations to enhance Alpha-Beta Pruning efficiency.	Board: current board state Player: identifier for agent Opponent: identifier for opponent Valid_moves: list of valid moves
evaluate_board	Assesses the desirability of the current board state for the agent using a pattern-based heuristic function.	Board: current board state Player: identifier for agent Opponent: identifier for opponent
get_adjacent_position	Retrieves all valid positions adjacent to a specified cell on the board.	r: row index of target cell c: column index of the target cell n: size of the Othello board
get_edges	Generates a list of all edge positions on the board, excluding the corners	n: size of the Othello board
count_stable_disks	Counts the number of stable disks for a specified player, where stable disks cannot be flipped.	Board: current board state Player: identifier for agent
count_frontier_disks	Counts the number of frontier discs for a specified player, which are more vulnerable to being flipped.	Board: current board state Player: identifier for agent
hash_board	Creates a unique hash of the current board state combined with the player's identifier for the transposition table.	Board: current board state Player: identifier for agent

6. Performance

The agent dynamically adjusts its search depth based on the size of the Othello board randomly varied by autoplay when simulating games. This adjustment ensures that the agent can perform deeper searches on smaller board where computational resources are less strained, and shallower searches on larger boards to maintain efficiency. Essentially, As the board size increases, the agent reduces the maximum search depth. This is a strategic decision to manage the exponential growth in the number of nodes due to the higher branching factor. We initialize max_depth and then implement the following specifications in the code:

- **Boards $\leq 6 \times 6$:** max_depth = 6
- **Boards 7×7 to 8×8 :** max_depth = 5
- **Boards 9×9 to 10×10 :** max_depth = 4
- **Boards $> 10 \times 10$:** max_depth = 3

We use iterative deepening to incrementally increase the depth from 1 to max_depth. in practice, the maximum search depth the agent reaches does depend on the board size. On smaller boards ($N \leq 8$) the agent can achieve a search depth up until the max depth of 6, while on larger boards the depth rarely gets above 3 (even without a max_depth ceiling). In the endgame when there are fewer disks on the board the depth achieved can grow massively due to more unidimensional searches, but the search function isn't harmed by maintaining the capped depth – the algorithm tends to perfectly compute at such stages

regardless. The depth in-game is not fixed for all moves. Some branches explore deeper than others based on move ordering and pruning, which helps the agent focus on the most relevant parts of the game tree.

We implement a capped `max_depth`, instead of limitless depth computed until the time limit for the sake of keeping computational requirements within predictable bounds, we found these reasonably capped depths didn't affect the performance at all. It also ensures that the memory usage for the transposition table is contained well under the 500mb constraints.

In terms of breadth, the number of moves considered at each level of the game varies. At higher depths, fewer moves are explored due to the increasing computational cost and exponential branching of the search tree. Move ordering and successful pruning is specifically designed to reduce the branching but this is pretty consistent between the max and the min player. I don't believe there is anything in the algorithm itself that would affect the breadth factor between the min and max player, especially having negamax implemented where the value of a game state for the current player is the negation of the value for the opponent. This means the same function is handling both the maximizing and minimizing decisions, so there will be no difference in the breadth between players. The only differences in breadth will arise from the dynamics of the game in progress itself.

In general, smaller boards have a higher search depth, lower branching factor and thus deeper planning and better performance in terms of move quality. Larger boards suffer from higher branching factors and lower search depth and will rely more on successful pruning and move ordering to maintain performance. There will inevitably be a drop off in performance as board sizes grow larger but this will be consistent over all other agents. The larger the board, the better implemented move-ordering and pruning approaches will have an advantage.

7. Estimated Results and Further Improvements

Testing our agent against the baseline models provided in the starter code has been good. The agent consistently beats `random_agent` and `gpt_greedy_corners` agent close to 100% of the time. To begin with, we made plenty of modified baseline models built off of the single ply look-ahead approach of the `gpt_agent`, but with enhanced heuristics. Our agent had no problem beating these with over 90% conviction, despite some of them being competitive against the early iterations of minimax and alpha beta with no additional features. I expect this agent to perform well against the class's agents, I don't believe our heuristic function has any glaring weak points that can be easily exploited, nor are there much stronger search strategies that could massively outperform ours given the allotted time constraint of 2 seconds. Perhaps an approach that doesn't have a `max_depth` limit and forcefully utilizes the full 2 seconds on every move will achieve higher depth than us, but whether this will translate to winning games is uncertain.

Where our agent will likely struggle the most, is the lack of dynamically varying the heuristic weights based on the game phase. In an early iteration, we split the weights into 3 phases of the game, defined by the ratio of disks applied to empty squares. The intuition is that clearly features like mobility and disk difference change in importance as the game progresses from early to late. We were unsuccessful in finding changing sets of weights and board emptiness splits that massively improved our performance. There are a couple reasons we believe this approach did not work. As a first, basic metrics like the number of empty squares or move counts might not capture the nuanced transitions between game phases. Different positional nuances are much more likely to be important than the raw arbitrary counts when determining game phase (In chess for example some openings are much more open/closed than others). Additionally, many of the intuitive heuristic weight changes occur in the endgame, where the win bonus may override any weight

changes, explaining why no change in performance was observed. Regardless, there is definitely potential to leverage different strategical features changing as the game phase progresses into a better evaluation function, the obvious next step in further developing our agent.

Works Cited:

https://courses.cs.washington.edu/courses/cse573/04au/Project/mini1/RUSSIA/miniproject1_vaishu_muthu/Paper/Final_Paper.pdf

<https://www.hanshq.net/othello.html#minimax>

<http://radagast.se/othello/howto.html#evaluation>

A world-Championship-level Othello Program – Paul Rosenbloom

<https://github.com/arminkz/Reversi/blob/master/README.md>

https://barberalec.github.io/pdf/An_Analysis_of_Othello_AI_Strategies.pdf